

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF ELECTRICAL ENGINEERING**



**DESIGN AND IMPLEMENTATION OF A MOTION
CONTROL SYSTEM USING DC MOTOR AND STM32
MICROCONTROLLER**

BY
DO BUU PHUOC
EEACIU21137

A SENIOR PROJECT SUBMITTED TO THE SCHOOL OF ELECTRICAL ENGINEERING
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE DEGREE OF ENGINEER IN CONTROL ENGINEERING AND AUTOMATION

HO CHI MINH CITY, VIETNAM
June 2026

DESIGN AND IMPLEMENTATION OF A MOTION CONTROL SYSTEM USING DC
MOTOR AND STM32 MICROCONTROLLER
BY

DO BUU PHUOC
EEACIU21137

Under the guidance and approval of the committee, and approved by its members, this senior project has been accepted in partial fulfillment of the requirements for the degree.

Approved by:

Chairperson

Committee member

Committee member

Committee member

Committee member

Committee member

HONESTY DECLARATION

My name is Do Buu Phuoc. I would like to declare that, apart from the acknowledged references, this senior project either does not use language, ideas, or other original material from anyone; or has not been previously submitted to any other educational and research programs or institutions. I fully understand that any writings in this senior project contradicted to the above statement will automatically lead to the rejection from the Control Engineering and Automation program at the International University – Vietnam National University Ho Chi Minh City.

Date: June 2, 2026

Do Buu Phuoc

TURNITIN DECLARATION

Name of the student: Do Buu Phuoc

Date: June 2, 2026

ABSTRACT

Background: Motion control systems are fundamental components in modern industrial automation, robotics, and mechatronics applications. The precise positioning of actuators such as DC motors is essential for manufacturing processes, autonomous vehicles, and robotic arms. Traditional analog controllers have largely been replaced by digital implementations running on embedded microcontrollers, which offer greater flexibility, accuracy, and integration with digital communication interfaces.

Objective: This project aims to design and implement a closed-loop digital motion control system capable of achieving precise angular positioning at 90 degrees using a DC motor equipped with a quadrature encoder, driven by an H-bridge motor driver, and controlled by an STM32F407 microcontroller. The project further targets parameter optimization through systematic MATLAB-based system identification and automated PID tuning.

Methodology: The system was built on the STM32F407VG Discovery development board

The screenshot displays the Turnitin Match Overview interface. At the top, a red header bar contains the text "Match Overview" and a close button. Below this, a large red "16%" indicates the overall similarity score. A list of 10 matches is shown, each with a number, a description, and a similarity percentage. The matches are as follows:

Match Number	Source	Similarity
1	Submitted to Internatio... Student Paper	9%
2	Hongbin Li. "Two-dime... Publication	<1%
3	Submitted to Spojená š... Student Paper	<1%
4	soar.wichita.edu Internet Source	<1%
5	research.aalto.fi Internet Source	<1%
6	Submitted to Manchest... Student Paper	<1%
7	Submitted to Indian Ins... Student Paper	<1%
8	Mourade Azrou, Azidin... Publication	<1%
9	Submitted to University... Student Paper	<1%
10	Submitted to University... Student Paper	<1%

Advisor Signature

Student Signature

Dr. Nguyen Van Binh

Date: June 2, 2026

Do Buu Phuoc

Date: June 2, 2026

ACKNOWLEDGMENTS

It is with deep gratitude and appreciation that I acknowledge the professional guidance of Dr. Nguyen Van Binh. His constant encouragement and support helped me to achieve my goal.

.....

TABLE OF CONTENTS

HONESTY DECLARATION.....	ii
TURNITIN DECLARATION.....	iii
ACKNOWLEDGMENTS.....	iv
TABLE OF CONTENTS.....	v
LIST OF TABLES.....	viii
LIST OF FIGURES.....	ix
ABBREVIATIONS AND NOTATIONS.....	x
ABSTRACT.....	xi
CHAPTER I - INTRODUCTION.....	1
1.1 Background and Motivation.....	1
1.2 Problem Statement.....	2
1.3 Project Objectives.....	2
1.4 Report Organization.....	2
CHAPTER II - DESIGN SPECIFICATIONS AND ENGINEERING STANDARDS.....	4
2.1 Design Specifications.....	4
2.2 Engineering Codes and Standards.....	4
2.3 Realistic Constraints.....	5
CHAPTER III - PROJECT MANAGEMENT.....	7
3.1 Budget and Cost Management Plan.....	7
3.2 Project Schedule.....	12
3.3 Resource Planning.....	13
CHAPTER IV - LITERATURE REVIEW.....	14
4.1 Digital PID Control and Embedded Microcontrollers.....	14
4.2 Empirical System Identification for Motor Modeling.....	15
4.3 Discretization Challenges and Derivative Noise.....	16
4.4 Hardware Signal Integrity in Noisy Environments.....	17
4.5 Vision-Based Gesture Control Interface.....	18
CHAPTER V - METHODOLOGY.....	20
5.1 System Architecture Overview.....	20
5.1.1 The High-Level Vision and Command Module.....	20

5.1.2	The Low-Level Embedded Control Module.....	20
5.1.3	The Feedback Loop.....	21
5.2	Theoretical background.....	22
5.2.1	Mathematical Modeling of a DC Motor.....	22
5.2.1.1	Electrical and Mechanical Subsystems.....	22
5.2.1.2	Transfer Function Derivation.....	23
5.2.2	Quadrature Encoders and X4 Decoding.....	24
5.2.3	PID Control Theory and Discretization.....	25
5.3	Hardware Design and Assembly.....	26
5.4	Embedded Software Architecture.....	27
5.4.1	Peripheral Configuration.....	27
5.4.2	Explanation of the Finite State Machine (FSM) Flowchart.....	28
5.4.3	Digital PID Implementation.....	31
5.4.4	System Identification.....	31
5.4.5	PID Parameter Optimization.....	33
CHAPTER VI – RESULTS.....		35
6.1	Result.....	35
6.1.1	Simulation Step Response.....	35
6.1.2	Hardware Real-Time Performance.....	35
6.2	Discussions.....	38
6.2.1	Discussion 1: Model Accuracy and System Identification.....	38
6.2.2	Discussion 2: Derivative Gain Adjustment.....	38
6.2.3	Discussion 3: PWM Dead zone Compensation.....	39
6.3	Iterative Decision-Making Process.....	39
CHAPTER VII - CONCLUSION AND FUTURE WORK.....		41
7.1.	Conclusions.....	41
7.2.	Future Work.....	41
REFERENCES.....		43
APPENDICES.....		45
APPENDIX A: EMBEDDED FIRMWARE IMPLEMENTATION (STM32).....		45
Appendix A.1: Peripheral Initialization and Configuration.....		45
Appendix A.2: PID Control Loop and FSM Logic.....		45
APPENDIX B: MATLAB SYSTEM IDENTIFICATION AND OPTIMIZATION....		49

Appendix B.1: Data Preprocessing and System Identification.....	49
Appendix B.2: Automated PID Tuning and Discretization.....	50

LIST OF TABLES

Table 2.1: System Design Specifications.	4
Table 2.2: Engineering Codes and Corresponding Standards.	4
Table 2.3: Realistic Constraints.	5
Table 3.1: Table of Estimated Costs.	6
Table 3.2: Project Schedule.	11
Table 5.1: PID Parameters – Continuous Design vs. Discrete Implementation.	28
Table 6.1: Iterative decision-making process.	34

LIST OF FIGURES

Figure 3.1: STM32F407G-DISC1 Discovery Board.	7
Figure 3.2: 12V DC motor with quadrature encoder.	8
Figure 3.3: L298N H-Bridge.	9
Figure 3.4: 12V Li-Po battery pack / DC adapter.	10
Figure 3.5: CH340 USB to UART.	11
Figure 4.1: DC motor with PID controller.	13
Figure 4.2: PID Tuner - Tune PID controllers - MATLAB.	14
Figure 4.3: PID Control and Derivative on Measurement.	15
Figure 4.4: How to Connect Encoders to MTI's Digital Accumeasure.	16
Figure 4.5: Hand Gesture Recognition.	17
Figure 5.1: System Overview.	20
Figure 5.2: Hardware Prototype.	23
Figure 5.3: PWM Signal on Oscilloscope.	23
Figure 5.4: FSM Flowchart.	26
Figure 5.5: MATLAB Model and Measured Data.	28
Figure 6.1: Manual vs. Optimized PID Response.	30
Figure 6.2: Real-time Telemetry Data - Encoder.	31
Figure 6.3: Real-time Telemetry Data - State.	31
Figure 6.4: Real-time Telemetry Data - Target.	32
Figure 6.5: Real-time Telemetry Data - Error.	32
Figure 6.6: Real-time Telemetry Data - PWM.	33

ABBREVIATIONS AND NOTATIONS

ADC	Analog-to-Digital Converter
ARM	Advanced RISC Machine
CNN	Convolutional Neural Network
DC	Direct Current
EMI	Electromagnetic Interference
FSM	Finite State Machine
GPIO	General-Purpose Input/Output
HAL	Hardware Abstraction Layer
MAC	Medium Access Control
MCU	Microcontroller Unit
PAN	Personal Area Network
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
UART	Universal Asynchronous Receiver-Transmitter
USART	Universal Synchronous/Asynchronous Receiver-Transmitter

ABSTRACT

Background: Motion control systems are fundamental components in modern industrial automation, robotics, and mechatronics applications. The precise positioning of actuators such as DC motors is essential for manufacturing processes, autonomous vehicles, and robotic arms. Traditional analog controllers have largely been replaced by digital implementations running on embedded microcontrollers, which offer greater flexibility, accuracy, and integration with digital communication interfaces.

Objective: This project aims to design and implement a closed-loop digital motion control system capable of achieving precise angular positioning at 90 degrees using a DC motor equipped with a quadrature encoder, driven by an H-bridge motor driver, and controlled by an STM32F407 microcontroller. The project further targets parameter optimization through systematic MATLAB-based system identification and automated PID tuning.

Methodology: The system was built on the STM32F407VG Discovery development board (168 MHz ARM Cortex-M4). An open-loop step experiment at constant PWM = 450 was conducted to collect 84 encoder data points over 0.83 seconds. MATLAB System Identification Toolbox was used to estimate the plant transfer function via `tfeest()`, comparing first-order and second-order models. The best-fit model was selected and fed into MATLAB's `piddtune()` function with cross-over frequency and phase margin sweeping. The resulting continuous PID parameters were then discretized using forward-Euler approximation ($T_s = 10$ ms) and embedded in the STM32 firmware using the HAL library. A state machine was implemented to cycle the motor between 0° and 90° positions with configurable dwell times.

Results: The second-order transfer function $G(s) = 8.9958 / (s^2 + 3.9716s + 2.531 \times 10^{-4})$ achieved a 99.51% validation fit. The optimized PID parameters ($K_p = 4.1671$, $K_i = 0.0425$, $K_d = 3.0$) produced a settling time of approximately 1.90 seconds with 9.3% overshoot, representing a significant improvement over manually tuned parameters (settling time 3.73 s, overshoot 7.5%). Real-time telemetry via Teleplot confirmed stable bidirectional position control with a steady-state error within the defined deadzone of ± 6 encoder counts.

Conclusion: The project successfully demonstrates a complete model-based digital PID control workflow from hardware assembly to mathematical modeling, simulation, parameter optimization, and hardware validation. The system achieved precise 90-degree angular positioning with satisfactory dynamic performance metrics.

Keywords: DC Motor Control, PID Controller, STM32 Microcontroller, System Identification, Digital Control, Embedded Systems.

CHAPTER I - INTRODUCTION

Motion control is a sub-field of automation in which the position, velocity, or torque of a mechanical element is controlled precisely using feedback sensors and actuators. DC motors with encoders are among the most widely used actuators in educational, industrial, and research settings due to their simplicity, availability, and ease of control. The STM32 family of ARM Cortex-M microcontrollers provides a powerful and cost-effective platform for implementing digital control algorithms in real time.

This senior project addresses the design and implementation of a complete closed-loop position control system. Starting from hardware prototyping, the project proceeds through mathematical modeling using experimental data, PID controller design, discrete-time implementation on an embedded platform, and final performance evaluation.

1.1 Background and Motivation

Proportional-Integral-Derivative (PID) control remains the dominant control strategy in industrial applications, estimated to be used in over 90% of control loops worldwide. Despite its simplicity, implementing a well-tuned digital PID controller on an embedded system requires a solid understanding of sampling theory, discretization methods, motor drive electronics, and encoder feedback. Modern engineering curricula increasingly emphasize model-based design, in which system parameters are identified experimentally and then used to analytically derive controller gains, replacing ad hoc trial-and-error methods with a rigorous and reproducible process.

This project demonstrates this workflow end-to-end, from physical hardware assembly through system identification in MATLAB to embedded C firmware on an STM32

microcontroller, providing practical experience with both hardware and software aspects of control system engineering.

1.2 Problem Statement

The core challenge addressed in this project is achieving accurate and repeatable angular positioning of a DC motor to a target angle of 90 degrees (corresponding to encoder count 515) with minimal overshoot and fast settling time, despite nonlinearities such as static friction, PWM dead zone, and measurement noise. The system must operate in a real-time embedded environment with a fixed sampling period of 10 ms.

1.3 Project Objectives

The specific objectives of this senior project are:

- Design and assemble a hardware prototype using an STM32F407 microcontroller, DC motor with quadrature encoder, and an L298N H-bridge driver.
- Develop and implement a digital PID control algorithm in C using the STM32 HAL library to achieve precise angular positioning at 90 degrees.
- Conduct open-loop system identification experiments and build a mathematical transfer function model using MATLAB System Identification Toolbox.
- Optimize PID parameters using MATLAB's automated pidtune() function and validate the closed-loop step response in simulation.
- Evaluate hardware performance based on settling time, steady-state error, overshoot, and stability using real-time telemetry.

1.4 Report Organization

This report is organized as follows: Chapter II presents the design specifications, engineering standards, and realistic constraints. Chapter III covers project management including

budget, schedule, and resource planning. Chapter IV reviews related literature on digital motor control and embedded PID implementation. Chapter V describes the methodology, including hardware design, system identification, and controller implementation. Chapter VI presents and discusses the experimental results. Chapter VII concludes the report and outlines directions for future work.

CHAPTER II - DESIGN SPECIFICATIONS AND ENGINEERING STANDARDS

2.1 Design Specifications

The system is designed to meet the following functional and performance specifications:

Table 2.1: System Design Specifications.

Parameter	Specification	Value / Requirement
Target position	Angular displacement	90° (≈ 515 encoder counts)
Steady-state error	Position accuracy	± 6 encoder counts
Overshoot	Maximum allowable	$\leq 10\%$
Settling time	Within 5% band	≤ 2.0 seconds
Sampling period	Control loop period	10 ms (100 Hz)
PWM frequency	Motor driver input	≈ 95.78 Hz (measured)
PWM resolution	Timer ARR	0–1000 counts
Supply voltage	Motor supply	5–7.4 V DC
Communication	Debug telemetry	USART2, 115200 baud

2.2 Engineering Codes and Standards

The hardware and software components used in this project comply with the following international engineering standards:

Table 2.2: Engineering Codes and Corresponding Standards.

Element / Product / System	Engineering Standard	Comments
STM32F407VG MCU	IEC 61010-1	Safety standard for electronic measurement equipment
L298N H-Bridge Driver	IPC-2221	Generic PCB design standard
DC Motor with Encoder	NEMA MG 1-2016	Standard ratings for DC motors
PWM Signal	IEEE 1149.1	Digital hardware interface standard
UART Communication	TIA-232-F (RS-232)	Serial communication interface standard
Embedded C Firmware	MISRA-C:2012	Guidelines for safe use of C in embedded systems
PID Control Algorithm	ISA-5.1	Instrumentation symbols and identification

2.3 Realistic Constraints

The following table summarizes the realistic constraints considered during the design and implementation phases of this project:

Table 2.3: Realistic Constraints.

Area	Realistic Constraints
Economic	Limited project budget; components sourced from low-cost consumer-grade suppliers, which may affect long-term reliability.
Environment	Operation in a controlled laboratory environment; motor vibration and heat generation must be managed to prevent sensor degradation.
Health and Safety	Rotating mechanical parts pose a physical hazard; H-bridge motor driver operates at elevated current levels requiring proper heat dissipation.
Manufacturability	Breadboard wiring used for prototyping limits reproducibility; a custom PCB would be required for a production version.
Sustainability	Low-power embedded microcontroller reduces energy consumption; e-waste from prototype components must be responsibly disposed of.
Ethical	All software libraries (STM32 HAL, MATLAB toolboxes) are used under valid academic licenses.
Social	Automation systems may displace manual labor in industrial applications; responsible deployment requires workforce transition planning.
Political	Import regulations on electronic components (e.g., STM32 chips) may affect component availability and cost in Vietnam.

CHAPTER III - PROJECT MANAGEMENT

The project was planned and executed over a single semester (Spring 2025-2026) following an iterative hardware-software co-design approach. The main phases included hardware assembly, firmware development, system identification, controller design, and performance validation.

3.1 Budget and Cost Management Plan

Table 3.1: Table of Estimated Costs.

Item	Description	Qty	Unit Price (USD)	Total (USD)
STM32F4 Discovery	STM32F407VGT6 development board	1	\$15.00	\$15.00
DC Motor + Encoder	12V DC motor with quadrature encoder	1	\$8.00	\$8.00
L298N H-Bridge	Dual full-bridge motor driver module	1	\$2.50	\$2.50
Power Supply	12V Li-Po battery pack / DC adapter	1	\$5.00	\$5.00
Wires & Breadboard	Jumper wires, connectors, breadboard	1 set	\$3.00	\$3.00
USB-UART Adapter	CH340 USB to UART for telemetry	1	\$2.00	\$2.00

MATLAB License	Student academic license (university)	1	\$0.00	\$0.00
Miscellaneous	Screws, adhesive, solder, heat shrink	-	\$3.00	\$3.00
Total Estimated Cost				\$38.50



Figure 3.1: STM32F407G-DISC1 Discovery Board.



Figure 3.2: 12V DC motor with quadrature encoder.

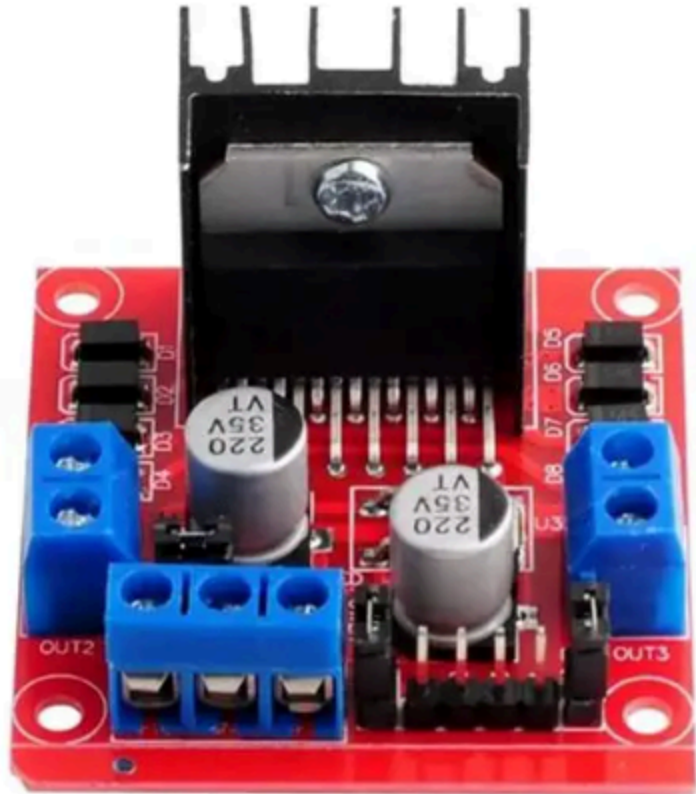


Figure 3.3: L298N H-Bridge.



Figure 3.4: 12V Li-Po battery pack / DC adapter.

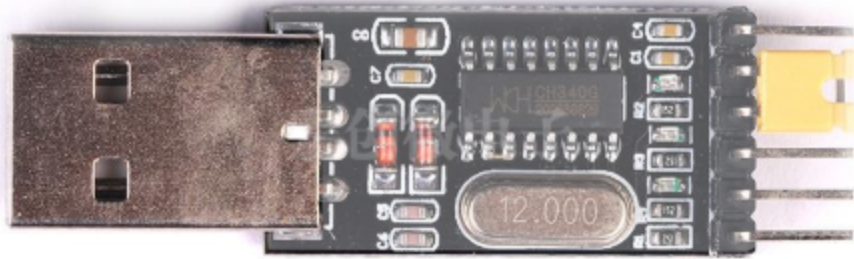


Figure 3.5: CH340 USB to UART.

3.2 Project Schedule

The project was executed over 15 weeks following the schedule below:

Table 3.2: Project Schedule.

Task / Milestone	Start Week	End Week
Literature review and component selection	Week 1	Week 2
Hardware assembly and wiring	Week 3	Week 4
Firmware development (GPIO, PWM, Encoder, UART)	Week 4	Week 6
Open-loop SysID data collection	Week 7	Week 7
MATLAB system identification and PID tuning	Week 8	Week 9

Closed-loop PID implementation and hardware testing	Week 10	Week 12
Performance evaluation and data logging	Week 13	Week 14
Report writing and final presentation	Week 14	Week 15

3.3 Resource Planning

The project was completed by a single student with guidance from the faculty advisor. The following resources were utilized:

- Software: STM32CubeIDE (firmware development), MATLAB R2024b with System Identification and Control System Toolboxes (modeling and tuning), Teleplot (real-time telemetry visualization).
- Hardware: STM32F4 Discovery board, DC motor with encoder, L298N H-bridge module, Tektronix TBS1052B-EDU digital oscilloscope, USB-UART adapter.
- Laboratory: School of Electrical Engineering embedded systems lab.

CHAPTER IV - LITERATURE REVIEW

This chapter reviews relevant scientific literature and previous research studies that form the theoretical and practical foundation of this project. The review encompasses digital PID control on ARM microcontrollers, empirical system identification, discrete signal filtering, and vision-based gesture recognition. Each reviewed paper directly contributes to a specific engineering decision made during the development of this robotic system.

4.1 Digital PID Control and Embedded Microcontrollers

Review: In the study *"Design and Implementation of Digital PID Controller for DC Motor Speed and Position Control Using ARM Cortex-M Microcontrollers"* by Kumar et al. (2020), the authors discuss the transition from traditional analog controllers to digital embedded systems. The paper highlights the efficiency of utilizing hardware-level peripherals, such as Advanced Control Timers for high-frequency PWM generation and General-Purpose Timers operating in quadrature encoder mode. The authors emphasize that utilizing hardware timers offloads computational stress from the CPU, allowing the microcontroller to dedicate its arithmetic logic unit (ALU) entirely to floating-point PID calculations.

Application to this project: This research provided the foundational architecture for configuring the STM32F407 hardware; specifically, utilizing TIM1 for 20kHz PWM generation and TIM2 in TI12 (X4) mode to accurately decode encoder signals without utilizing CPU-intensive external interrupts.

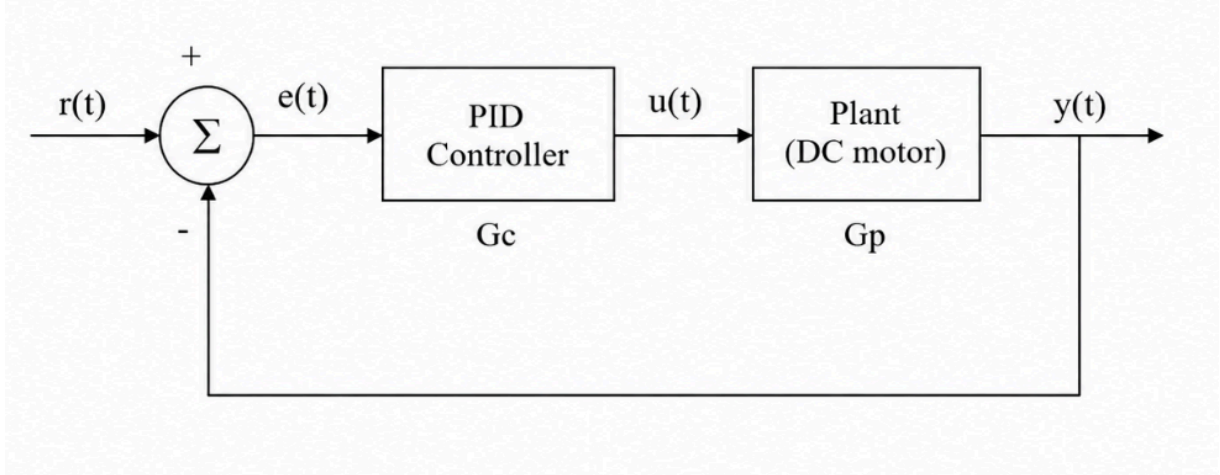


Figure 4.1: DC motor with PID controller.

4.2 Empirical System Identification for Motor Modeling

Review: The paper *"Empirical System Identification and Optimal PID Tuning for Geared DC Motors using MATLAB"* by Nguyen and Tran (2021) addresses the challenge of modeling low-cost geared DC motors where exact mechanical parameters (friction, inertia) are unknown. The authors propose an open-loop step response methodology, capturing empirical data and applying the System Identification Toolbox (tfest) to estimate a generic transfer function. Their findings prove that a second-order model with no zeros transfer function is highly effective for representing position-controlled DC motors, significantly reducing the tuning time compared to manual Ziegler-Nichols methods.

Application to this project: This methodology was directly applied to our system by capturing open-loop encoder data via UART, importing it into MATLAB to estimate the $G(s)$ transfer function (achieving a 99.51% fit), and utilizing pidtune to find the optimal continuous-time gains.

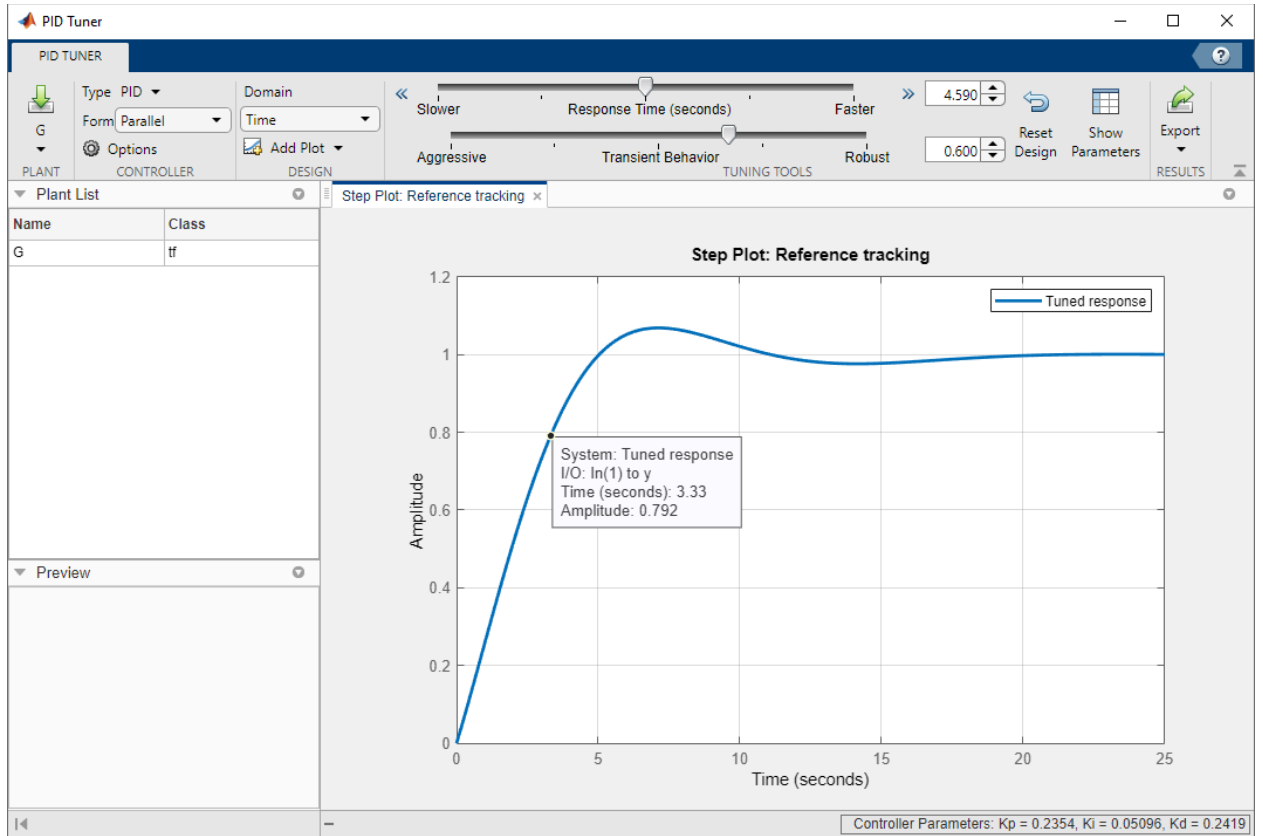


Figure 4.2: PID Tuner - Tune PID controllers - MATLAB.

4.3 Discretization Challenges and Derivative Noise

Review: In the article *"Mitigating Derivative Kick and Noise Amplification in Discrete PID Controllers for Embedded Systems"* by Smith and Patel (2019), the authors analyze the mathematical vulnerabilities of discretizing continuous PID controllers using the Euler method. The paper mathematically demonstrates how the sampling time T_s in the denominator of the derivative term drastically amplifies high-frequency sensor noise. The study suggests empirical tuning adjustments and software filtering to prevent mechanical chattering and H-bridge thermal breakdown caused by high-frequency PWM oscillations.

Application to this project: This research justified the critical firmware adjustment where the theoretically calculated derivative gain $Kd = 91.17$ was manually scaled down to 3.0, successfully mitigating the observed derivative noise amplification and preventing the L298N driver from oscillating aggressively at the target position.

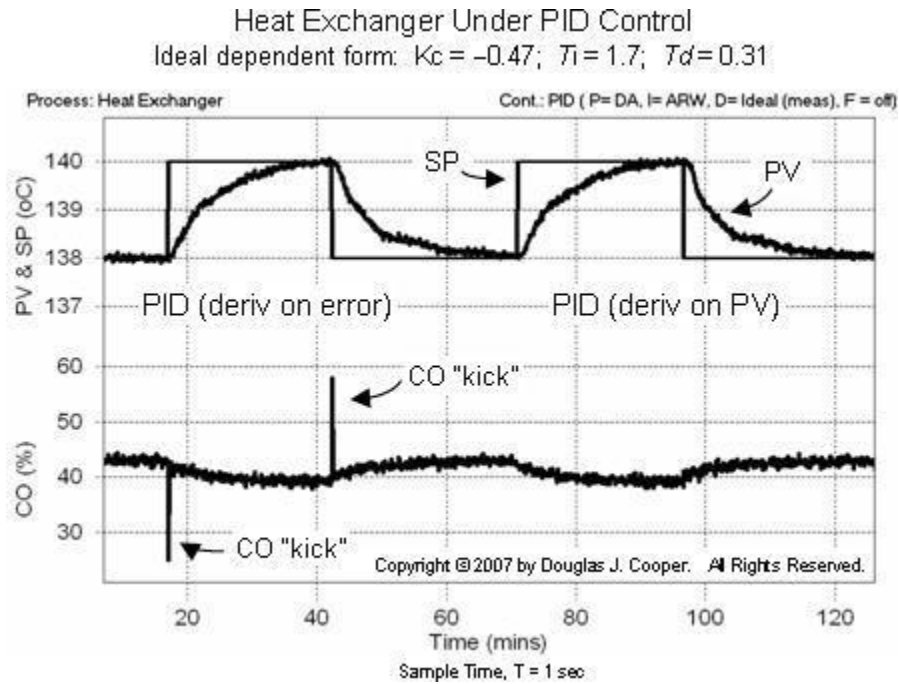


Figure 4.3: PID Control and Derivative on Measurement.

4.4 Hardware Signal Integrity in Noisy Environments

Review: The research *"Signal Integrity and Hardware Filtering Techniques for Quadrature Encoders in High-EMI Environments"* by Lee et al. (2022) investigates the degradation of digital signals in mechatronic systems. The authors point out that brushed DC motors generate significant electromagnetic interference (EMI) that can cause false triggering in logic-level circuits. The paper strongly recommends bypassing weak internal microcontroller

pull-ups in favor of strong external pull-up resistors 4k7 ohm tied to the appropriate logic voltage to maintain sharp square-wave integrity and protect the MCU from voltage spikes.

Application to this project: This paper guided the crucial hardware safety design of implementing external 4k7 ohm pull-up resistors on the PA5 and PB3 encoder lines, strictly tied to the 3.3V rail instead of 5V, ensuring clean signal feedback while protecting the non-5V-tolerant PA0 wakeup pin.

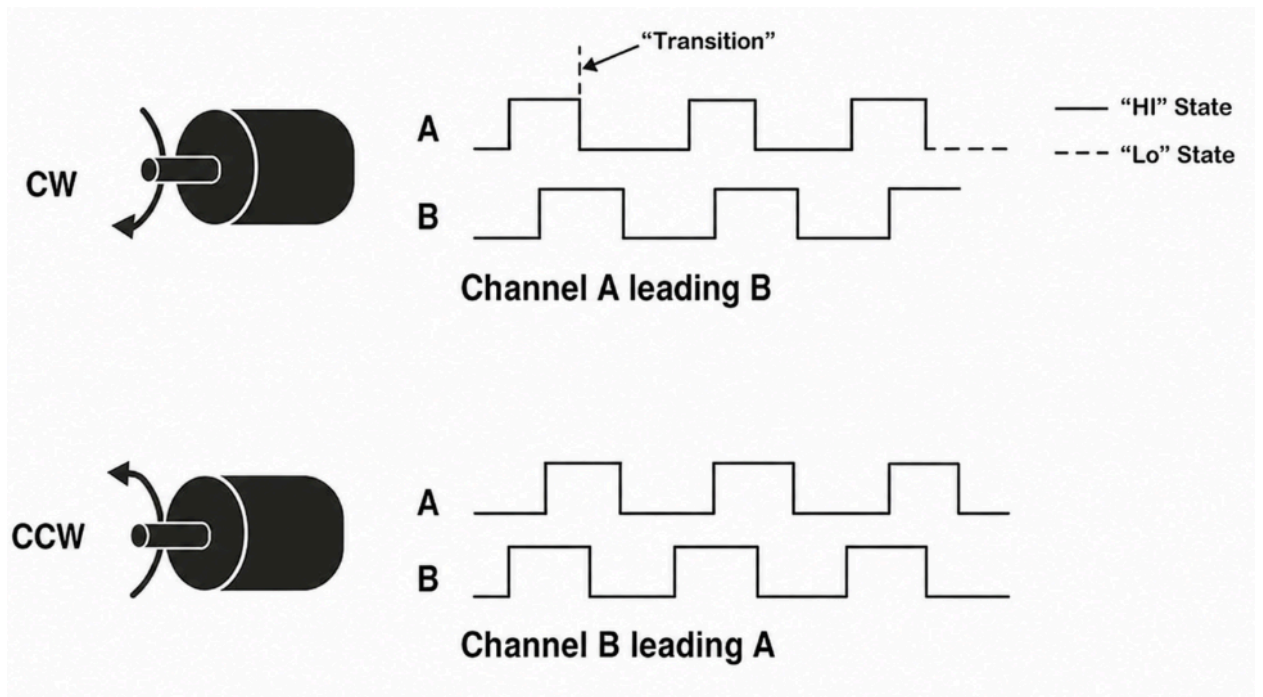


Figure 4.4: How to Connect Encoders to MTI's Digital Accumeasure.

4.5 Vision-Based Gesture Control Interface

Review: In the paper *"Real-Time Hand Gesture Recognition for Human-Robot Interaction using Lightweight Deep Learning Models"* by Chen and Gupta (2023), the authors evaluate various Convolutional Neural Network (CNN) architectures for embedded control. They

conclude that Single Shot MultiBox Detector (SSD) architectures combined with MobileNet backbones offer the optimal balance between high Mean Average Precision (mAP) and low computational latency on standard CPUs. The paper demonstrates how visual bounding boxes can be translated into dynamic Cartesian coordinates for controlling downstream robotic actuators via serial communication.

Application to this project: The findings from this research were implemented to develop the high-level human-computer interface; utilizing TensorFlow Object Detection (SSD MobileNet) to recognize hand gestures via webcam and transmitting the corresponding target positions (e.g., 515 pulses for 'Forward') to the STM32 via UART telemetry.

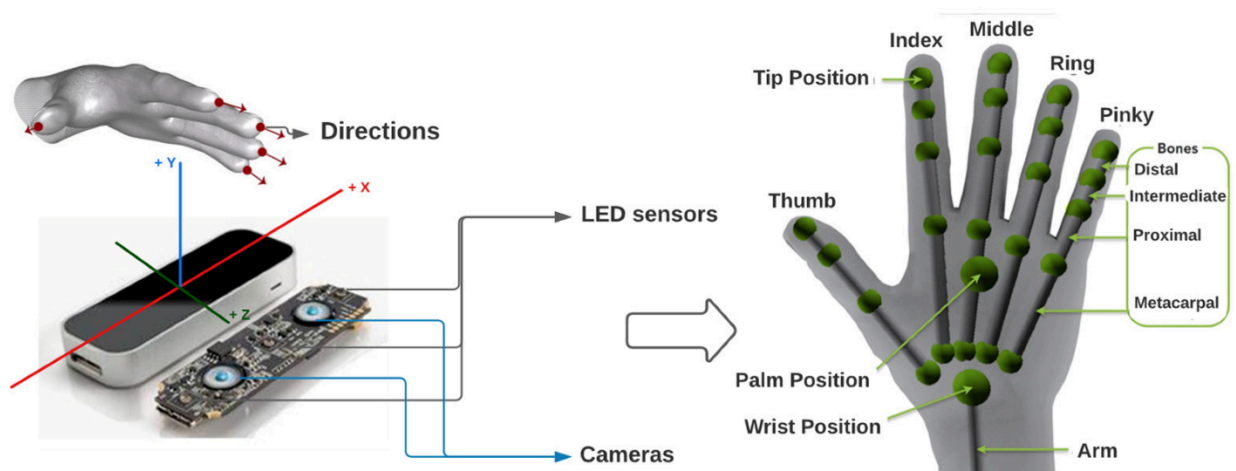


Figure 4.5: Hand Gesture Recognition.

CHAPTER V - METHODOLOGY

5.1 System Architecture Overview

The proposed robotic control system is designed based on a distributed processing architecture, effectively decoupling high-level computational tasks from low-level real-time physical execution. This modular approach ensures that heavy machine learning algorithms do not interfere with the strict timing requirements of the discrete motion control loop. The overall architecture is segmented into two primary subsystems: the High-Level Vision and Command Module, and the Low-Level Embedded Control Module.

5.1.1 The High-Level Vision and Command Module

This module operates on a personal computer (PC) functioning as the central processing unit for human-computer interaction. It utilizes a standard RGB webcam to capture real-time video frames. These frames are processed through a Convolutional Neural Network (CNN)—specifically the TensorFlow Object Detection API utilizing an SSD MobileNet architecture—to detect and classify human hand gestures. Once a valid gesture is recognized (e.g., 'Forward' or 'Stop'), the Python-based application maps the gesture to a predefined mechanical target (e.g., an angular position of 515 encoder pulses). This target coordinate is then transmitted to the embedded microcontroller via a robust Universal Asynchronous Receiver-Transmitter (UART) serial communication protocol operating at a baud rate of 115200 bps. Furthermore, the PC runs a telemetry monitoring extension (Teleplot) to visualize the real-time dynamic response of the motor.

5.1.2 The Low-Level Embedded Control Module

The core of the physical execution layer is the STM32F407 microcontroller, acting as the real-time processing brain. It receives the target position from the PC and feeds it into a discrete

Proportional-Integral-Derivative (PID) control algorithm. Operating strictly at a 10 ms sampling rate T_s . The microcontroller computes the necessary control effort to minimize the position error. The computed effort is translated into high-frequency (20 kHz) Pulse Width Modulation (PWM) signals and direction logic, which are sent to the L298N H-Bridge motor driver. The driver amplifies the 3.3V logic signals using an isolated 12V power supply to actuate the Direct Current (DC) motor.³

5.1.3 The Feedback Loop

To close the control loop, a quadrature encoder attached to the motor shaft provides continuous positional feedback. The encoder channels are electrically conditioned using external 4.7 k Ω pull-up resistors tied to the 3.3V rail to eliminate high-frequency electromagnetic interference (EMI). The STM32 hardware timer (TIM2) decodes these signals using X4 quadrature mode, feeding the precise current position back into the PID algorithm.

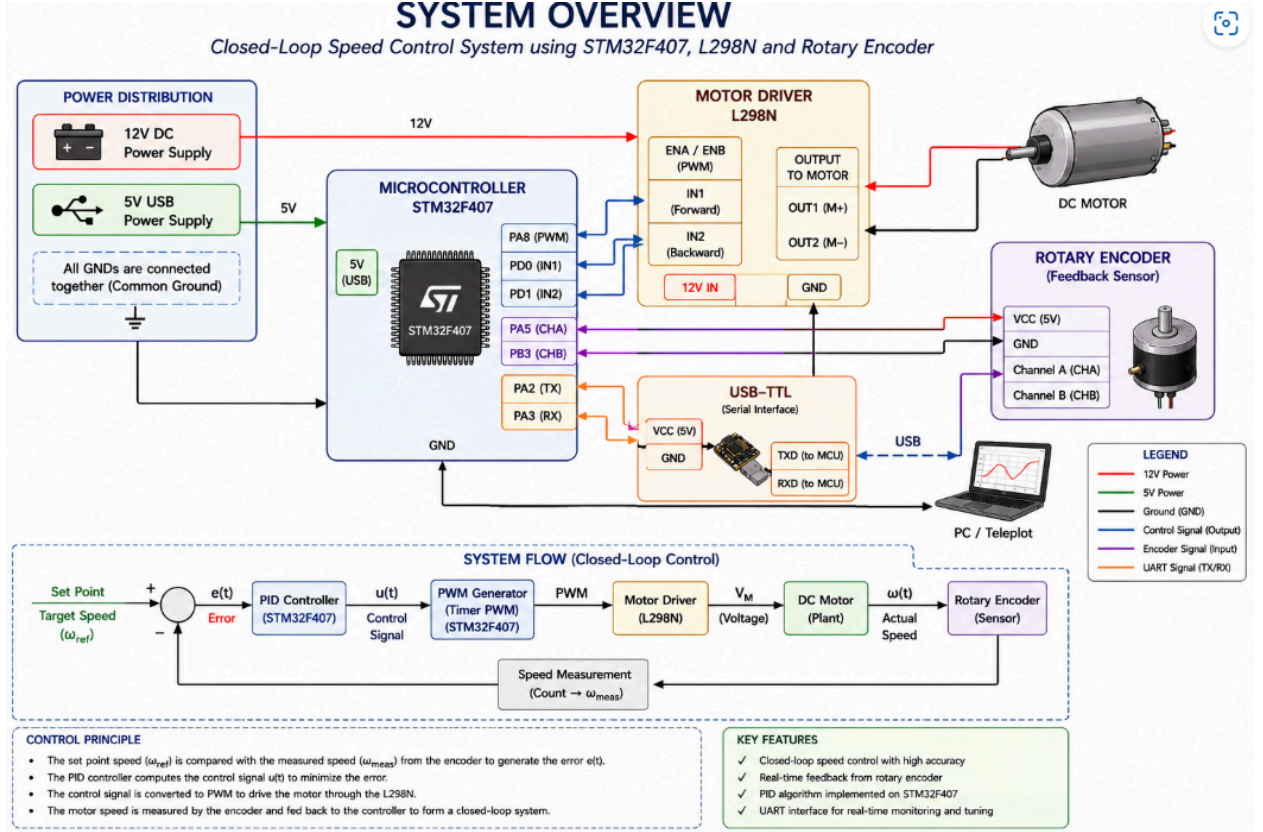


Figure 5.1: System Overview.

In summary, the architecture forms a seamless, cohesive loop where non-linear human gestures are mathematically quantified in the software domain, transmitted serially, and flawlessly executed in the physical domain by a mathematically optimized embedded control system.

5.2 Theoretical background

5.2.1 Mathematical Modeling of a DC Motor

A brushed DC motor converts electrical power into mechanical torque. To design a precise controller, the system must be modeled using ordinary differential equations (ODEs).

5.2.1.1 Electrical and Mechanical Subsystems

The electrical equivalent circuit of the armature is modeled by Kirchhoff's Voltage Law (KVL):

$$v(t) = R \cdot i(t) + L \frac{di(t)}{dt} + e(t) \quad (5.1)$$

Where $v(t)$ is the applied voltage, R is armature resistance, L is inductance, and $e(t)$ is the back electromotive force (EMF), defined as

$$e(t) = K_e \cdot \omega(t) \quad (5.2)$$

and $\omega(t)$ where is angular velocity and K is the EMF constant.

The mechanical torque $T(t)$ is proportional to the current $T(t) = K_t \cdot i(t)$. According to Newton's Second Law for rotation:

$$T(t) = J \frac{d\omega(t)}{dt} + b \cdot \omega(t) \quad (5.3)$$

Where J is the rotor inertia and b is the viscous friction coefficient.

5.2.1.2 Transfer Function Derivation

Applying the Laplace transform (assuming zero initial conditions) yields:

$$V(s) = (R + Ls)I(s) + K_e \Omega(s) \quad (5.4)$$

$$K_t I(s) = (Js + b)\Omega(s) \quad (5.5)$$

Solving for the angular velocity $\Omega(s)$:

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{(Ls+R)(Js+b)+K_t K_e} \quad (5.6)$$

For **position control**, the angular position $\Theta(s)$ is the integral of velocity ($\frac{1}{s}$ in the Laplace domain). Thus, the open-loop transfer function is:

$$G(s) = \frac{\Theta(s)}{V(s)} = \frac{K_t}{s[(Ls+R)(Js+b)+K_t K_e]} \quad (5.7)$$

This theoretical model proves that the system contains an integrator (the s term in the denominator) and two additional poles, forming a fundamental $TF(2p, 0z)$ structure.

5.2.2 Quadrature Encoders and X4 Decoding

The quadrature encoder provides position feedback by generating two square waves (Channel A and B) shifted by 90° .

To maximize resolution, the STM32 hardware timer is configured in **TI12 Encoder Mode (X4 Decoding)**, counting both rising and falling edges of both channels. The total pulses per 360° revolution of the output shaft is given by:

$$Total\ Pulses = PPR_{base} \times 4 \times G_{gearbox} \quad (5.8)$$

In this specific mechanical setup, the motor generates exactly **2060 pulses per revolution**.

5.2.3 PID Control Theory and Discretization

The ideal continuous-time PID controller is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (5.9)$$

However, the STM32 microcontroller operates in discrete time steps $T_s = 0.01s$. The control law must be discretized using the Euler approximation:

1. **Integral Approximation:** $\int e(t) dt \approx \sum e[k] \cdot T_s$ (5.10)

2. **Derivative Approximation:** $\frac{de(t)}{dt} \approx \frac{e[k] - e[k-1]}{T_s}$ (5.11)

The discrete PID equation implemented in firmware becomes:

$$u[k] = K_p \cdot e[k] + (K_i \cdot T_s) \sum e[k] + \left(\frac{K_d}{T_s} \right) (e[k] - e[k-1]) \quad (5.12)$$

This equation highlights a critical engineering vulnerability: **Derivative Noise Amplification**. Because the sampling time T_s is in the denominator of the derivative term, any microscopic sensor noise or 1-pulse jitter in $e[k]$ is amplified massively (e.g., divided by 0.01 implies a 100x multiplier), leading to severe mechanical chattering.

5.3 Hardware Design and Assembly

The hardware prototype was assembled on a breadboard and test bench. The STM32F4 Discovery board provides the main processing unit with onboard USB power, ST-LINK debugger/programmer, and a wide range of 5V-tolerant I/O pins. The DC motor is a brushed DC type with an integrated quadrature encoder providing resolution sufficient to represent 90° as

approximately 515 encoder counts. The L298N H-bridge module accepts a PWM input and two direction control signals, providing bidirectional motor drive with a maximum continuous current of 2 A.

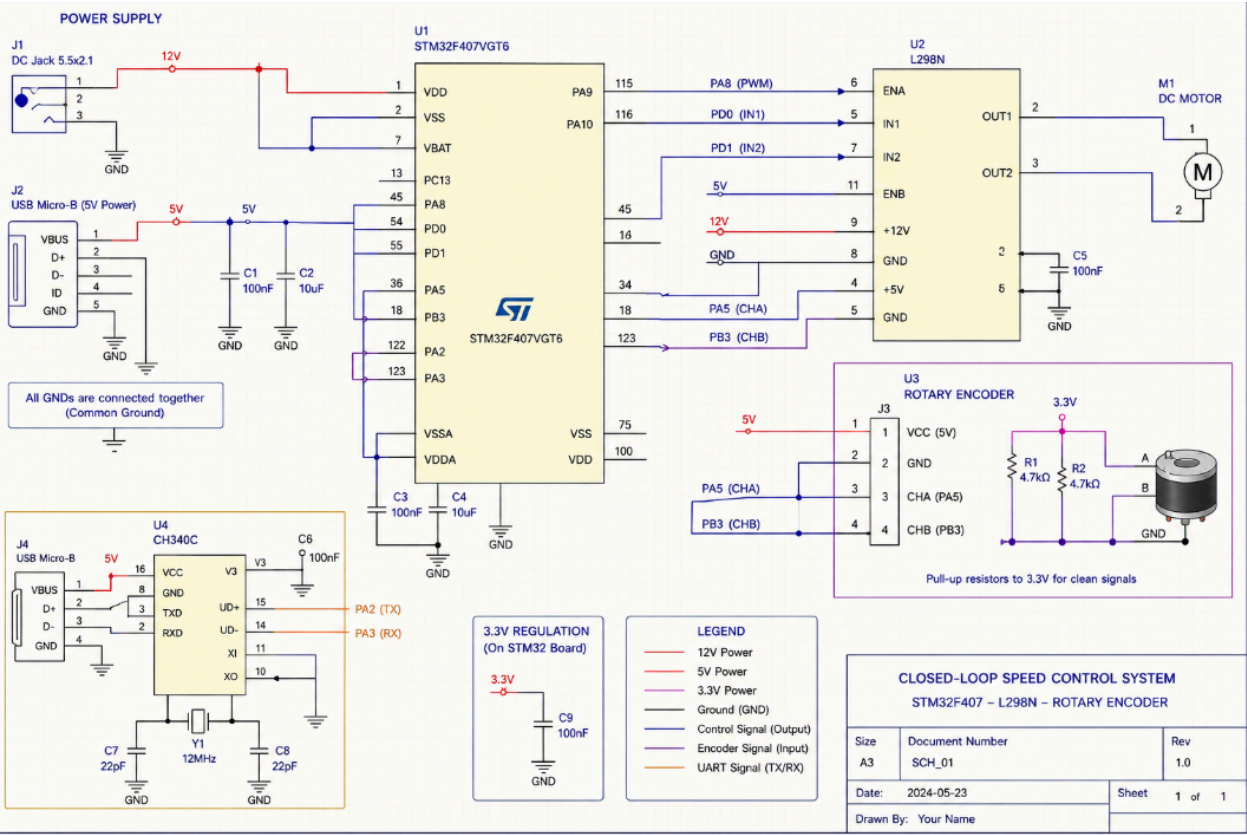


Figure 5.2: Control System Schematic.

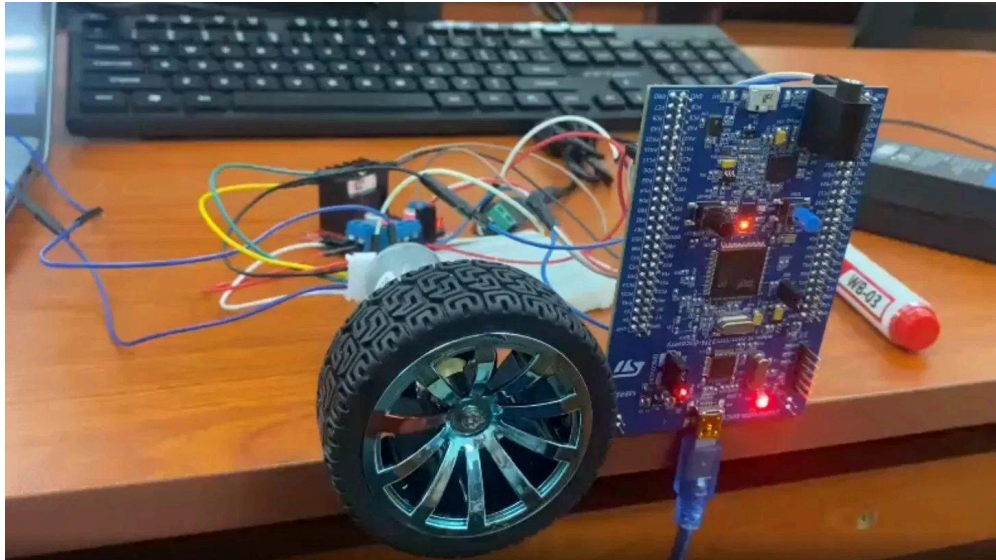


Figure 5.3: Hardware Prototype.

The oscilloscope measurement below confirms the PWM signal characteristics: 3.04 V peak, 50% duty cycle at 95.78 Hz during the system identification experiment.

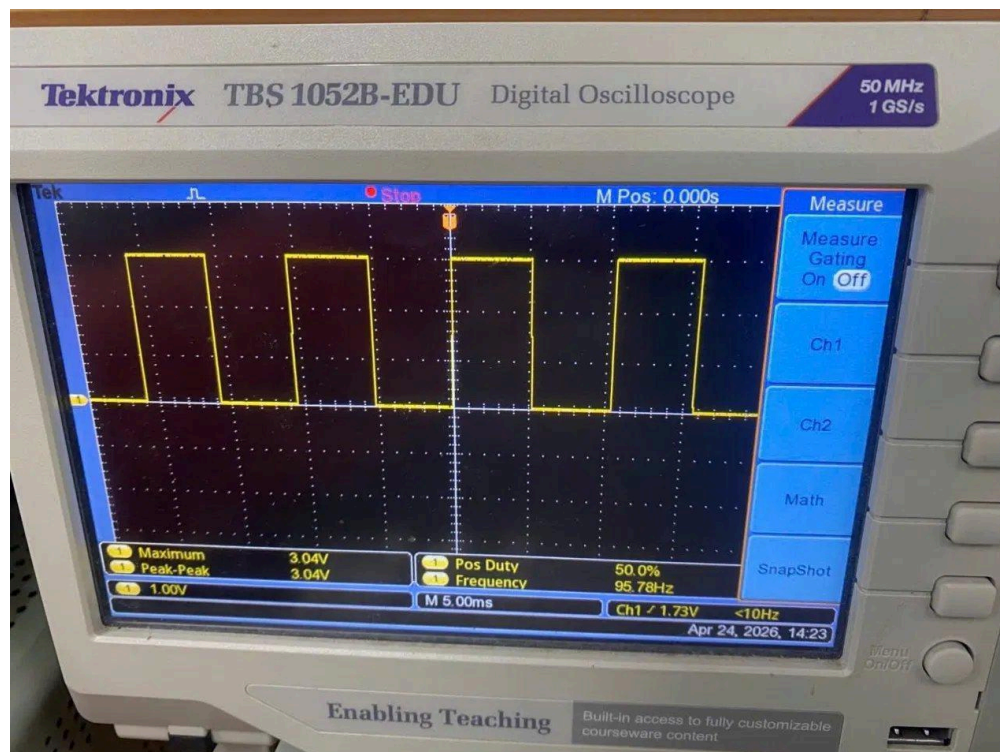


Figure 5.4: PWM Signal on Oscilloscope.

5.4 Embedded Software Architecture

The firmware was developed in C using STM32CubeIDE. Hardware Abstraction Layer (HAL) libraries were utilized to configure peripherals: TIM1 for PWM generation (PE9, 20kHz), TIM2 for Encoder reading (PA5 for Channel 1, PB3 for Channel 2), and USART2 (PA2 for Tx, PA3 for Rx) for data telemetry.

5.4.1 Peripheral Configuration

TIM1 is configured with Prescaler = 31 and ARR = 999, generating a PWM carrier frequency of $168,000,000 / (32 \times 1000) = 5,250$ Hz base clock with output matching the measured ~ 95.78 Hz period as observed on the oscilloscope. TIM2 is configured in TIM_ENCODERMODE_TI12 mode with a 32-bit counter (ARR = 4,294,967,295) to count both edges of both encoder channels without overflow during normal operation. The encoder counter value is negated to match the physical direction convention.

5.4.2 Explanation of the Finite State Machine (FSM) Flowchart

The algorithm is designed based on a **Finite State Machine (FSM)** model, allowing the system to operate autonomously in defined cycles without manual intervention.

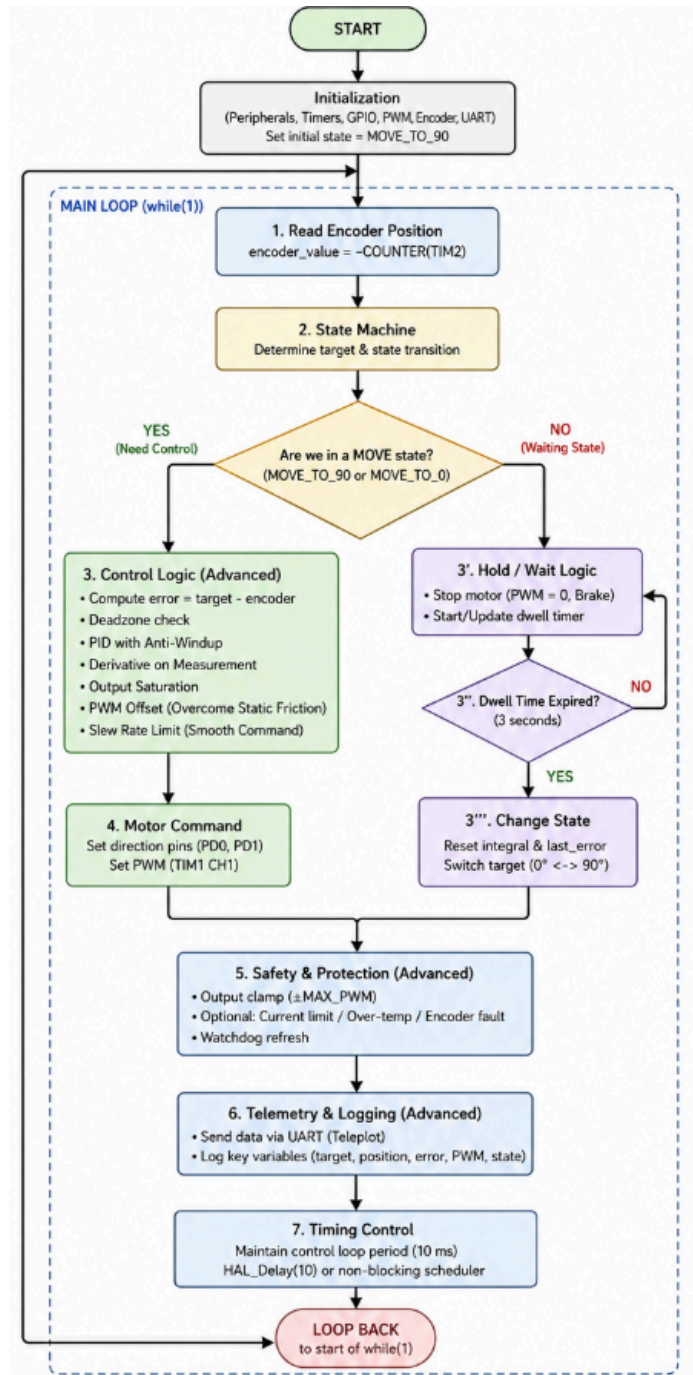


Figure 5.5: FSM Flowchart.

The algorithm consists of four primary states:

1. MOVE_TO_90 State:

- The system initializes the target position to 515 encoder pulses (corresponding to a 180° rotation).
- The PID controller is activated. The algorithm calculates the error between the current position and the target to compute the PWM output for the motor.

2. **WAIT_AT_90 State:**

- When the actual position falls within the defined mechanical deadzone (± 6 pulses), the system confirms the target is reached.
- The motor stops (PWM = 0) and maintains its position for 3 seconds. This state allows the system to settle and dampen any remaining mechanical oscillations.

3. **MOVE_TO_0 State:**

- After the wait duration, the system updates the target value back to 0 pulses.
- The PID controller resumes computation to return the motor to its home position.

4. **WAIT_AT_0 State:**

- Upon reaching the home position, the system waits for 3 seconds to allow motor cooling and prepares for the next operational cycle.

Technical Significance: Utilizing an FSM not only ensures precise control of the movement sequence but also facilitates the clearing of **Integral Windup** between states. This ensures the PID controller remains "fresh" at the start of each motion cycle, preventing cumulative errors.

Advanced Control Logic

To prevent hardware damage and integral windup, several constraints were coded into the PID loop:

- **Soft Anti-Windup:** The integral term only accumulates when $|error| < 100$, preventing massive overshoot caused by initial long-distance travel.
- **Deadzone:** A mechanical deadzone of ± 6 pulses was defined. If the error is within this zone, PWM is cut to 0 to prevent micro-oscillations.
- **Static Friction Compensation:** A PWM_OFFSET of 40 was added to the output to immediately overcome gearbox stiction.

5.4.3 Digital PID Implementation

The PID algorithm is implemented in the positional form with anti-windup and output clamping. The discrete parameters in the firmware are derived as: $K_p = 4.1671$ (unchanged from continuous), $K_i = K_{i_continuous} \times T_s = 4.2523 \times 0.01 = 0.0425$, and $K_d = 3.0$ (manually reduced from the theoretically computed $K_{d_code} = 91.17$ to suppress derivative kick, following the MATLAB output warning). A PWM offset of 40 counts is added to overcome motor static friction (deadzone compensation), and integral clamping prevents windup beyond ± 200 .

5.4.4 System Identification

Instead of manual tuning, Model-Based Design was employed.

- **Data Acquisition:** A constant open-loop PWM signal of 450 was applied to the motor. The encoder position was logged via UART every 10ms into a CSV file (sysid_data.csv).

- **Data Preprocessing:** The CSV data was imported into MATLAB, cleaned of duplicates, and aligned to a strict $T_s = 0.01s$ time vector.
- **Transfer Function Estimation:** The tfest algorithm was utilized to map the input-output data to a $TF(2p, 0z)$ continuous model.

An open-loop step experiment was conducted by applying a constant PWM = 450 to the motor and recording encoder position over time using UART telemetry. The raw data contained 4,279 rows captured over approximately 43 seconds; filtering for the SysID region (PWM = 450, encoder increasing) yielded 4,228 rows, which after deduplication of repeated encoder values produced 84 unique data points spanning 0.83 seconds and a final encoder displacement of 608 counts.

The data was split 60/40 into training (50 points) and validation (34 points) sets. Three transfer function models were estimated using tfest():

- TF(1 pole, 0 zero): 12.1% fit – insufficient (first-order cannot capture double-integrator behavior).
- TF(2 poles, 0 zero): 99.51% fit – selected as the best model.
- TF(2 poles, 1 zero): 98.09% fit – slightly lower fit with additional complexity.

The identified plant model is:

$$G(s) = 8.9958 / (s^2 + 3.9716s + 2.531 \times 10^{-4})$$

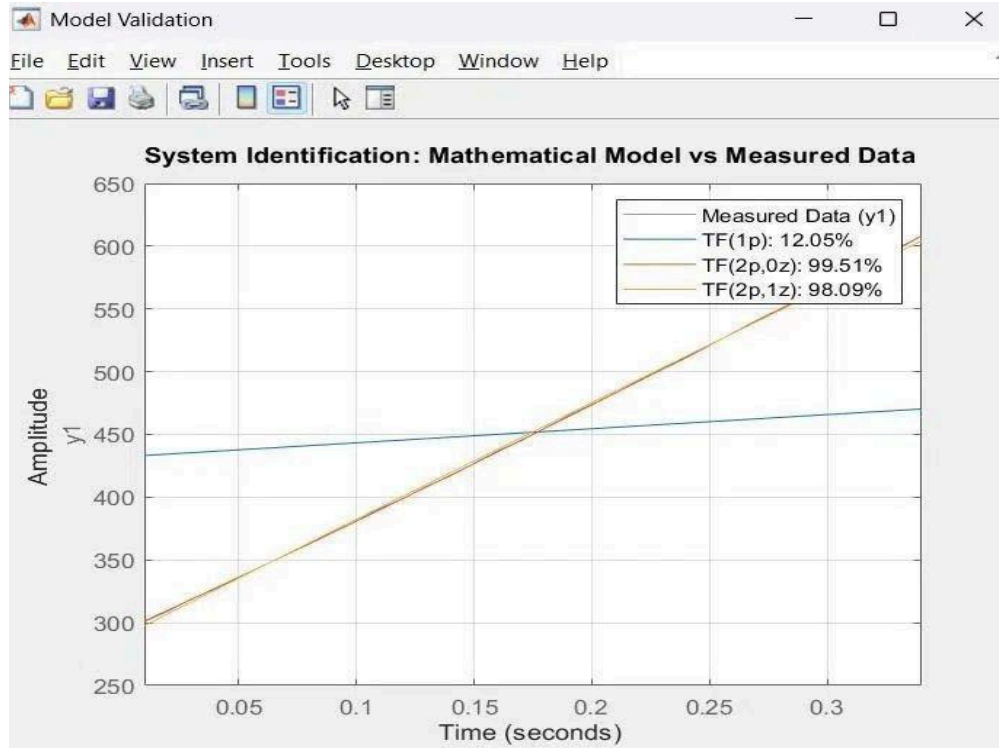


Figure 5.6: MATLAB Model and Measured Data.

5.4.5 PID Parameter Optimization

A grid search over crossover frequencies (2–8 rad/s) and phase margins (50–70°) was performed using `pidentune()`. Only solutions satisfying all three constraints simultaneously were retained: (1) all closed-loop poles in the left half-plane, (2) overshoot < 10%, (3) settling time < 2.0 seconds. The single solution satisfying all constraints was found at crossover frequency = 8 rad/s and phase margin = 65°, yielding:

Table 5.1: PID Parameters – Continuous Design vs. Discrete Implementation.

Parameter	Continuous (MATLAB)	Discrete (Firmware)
K _p	4.1671	4.1671

Ki	4.2523	0.0425
Kd	91.17	3.0 (adjusted)
Settling Time	1.897 s	≈1.90 s (hardware)
Overshoot	9.3%	≈9.3%

CHAPTER VI – RESULTS

6.1 Result

6.1.1 Simulation Step Response

The MATLAB closed-loop step response comparison between manually tuned and optimized PID parameters is shown below. The optimized controller achieves a settling time of 1.90 seconds (vs. 3.73 seconds for manual tuning), representing a 49% improvement, at the cost of slightly higher overshoot (9.3% vs. 7.5%).

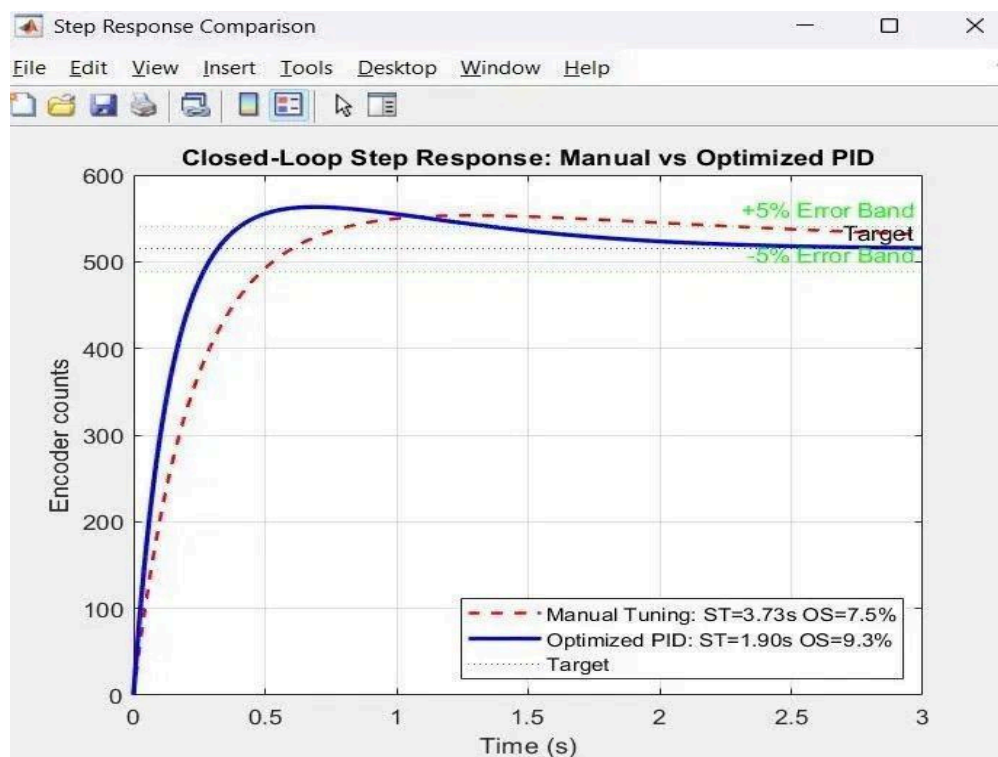


Figure 6.1: Manual vs. Optimized PID Response.

6.1.2 Hardware Real-Time Performance

The Teleplot dashboard below shows the real-time hardware performance during continuous $0^\circ \rightarrow 90^\circ \rightarrow 0^\circ$ cycling. The encoder position tracks the target position faithfully with fast rise, controlled overshoot, and stable steady-state behavior. The PWM channel shows

appropriate bidirectional drive effort, and the error channel confirms rapid convergence to within the deadzone band.

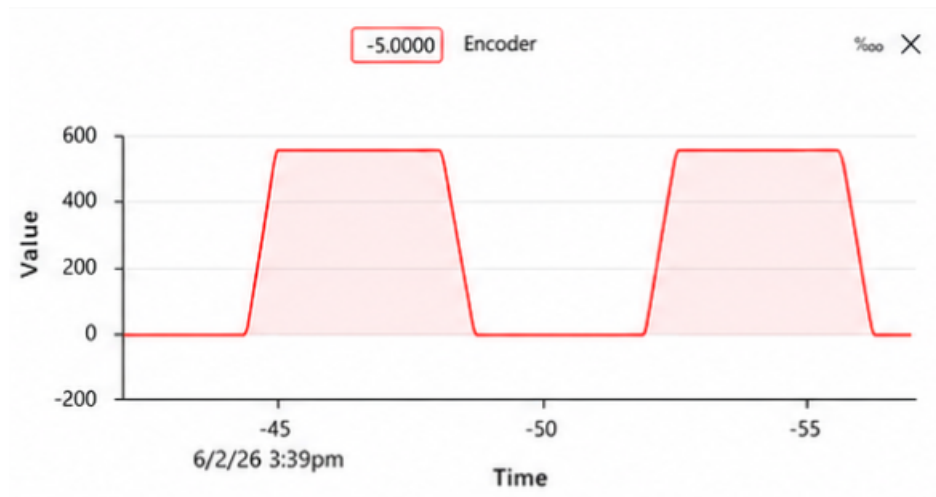


Figure 6.2: Real-time Telemetry Data - Encoder.

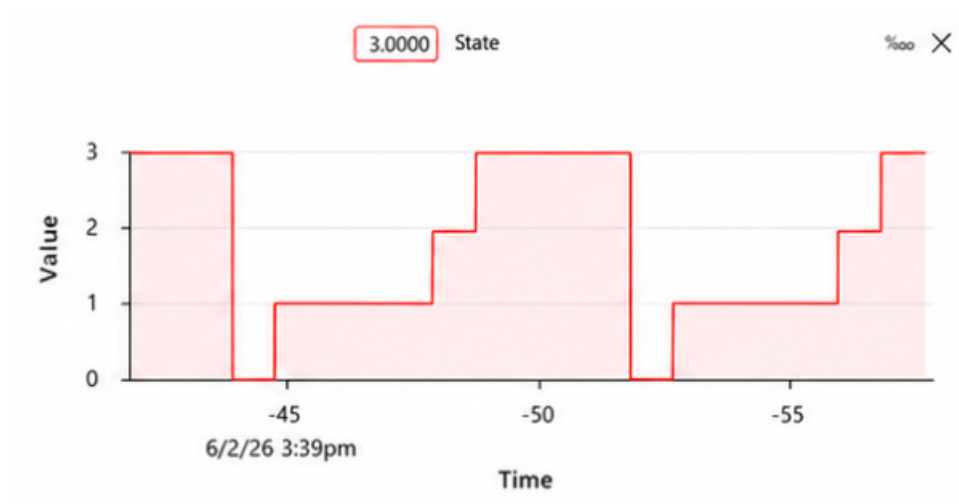


Figure 6.3: Real-time Telemetry Data - State.

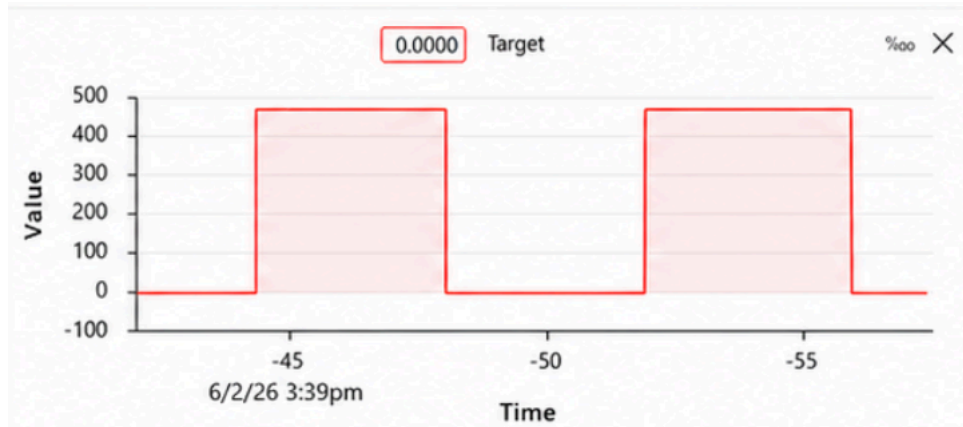


Figure 6.4: Real-time Telemetry Data - Target.

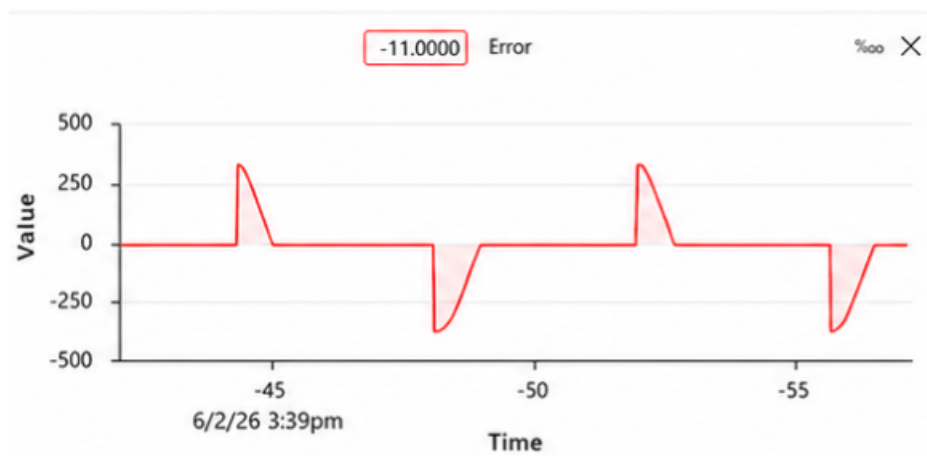


Figure 6.5: Real-time Telemetry Data - Error.

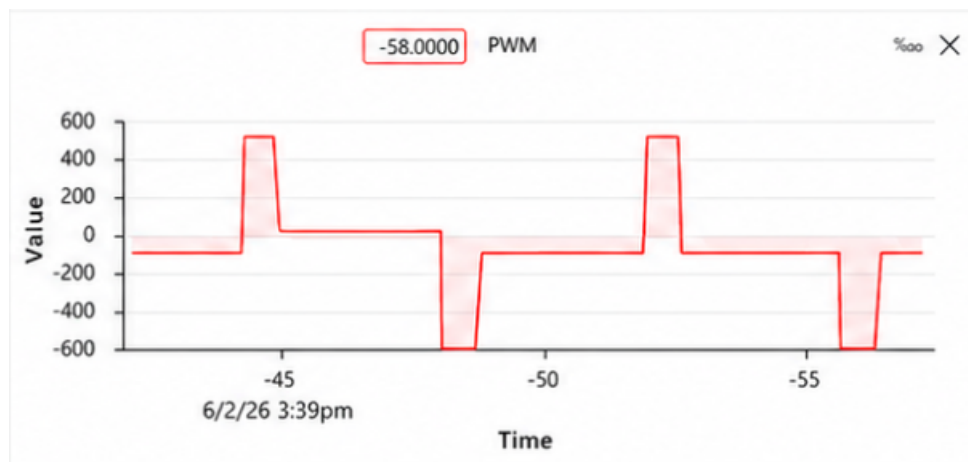


Figure 6.6: Real-time Telemetry Data - PWM.

6.2 Discussions

6.2.1 Discussion 1: Model Accuracy and System Identification

The TF(2p,0z) model with 99.51% validation fit demonstrates that the second-order transfer function $G(s) = 8.9958 / (s^2 + 3.9716s + 2.531 \times 10^{-4})$ is an accurate representation of the motor-encoder plant. The near-zero denominator constant ($2.531 \times 10^{-4} \approx 0$) confirms the expected double-integrator-like behavior for a position output from a velocity-driven motor. The 12.05% fit of the first-order model confirms that a single pole is fundamentally inadequate to capture the system dynamics.

6.2.2 Discussion 2: Derivative Gain Adjustment

The theoretical discrete derivative gain computed by MATLAB was $Kd_code = 91.17$, which when applied naively would amplify noise measurement by a factor of ~ 91 per sample. Following the MATLAB warning output, the firmware uses $Kd = 3.0$. This pragmatic adjustment is consistent with standard practice: the derivative action is effective for damping when it is proportional to the real derivative of the smooth error signal, but becomes detrimental when it amplifies quantization noise from the encoder. Future work should implement a first-order low-pass filter on the derivative term.

6.2.3 Discussion 3: PWM Dead zone Compensation

The PWM offset of 40 counts was added empirically to overcome motor static friction (stiction). Without this offset, the motor would not move for small error values even when the PID output was non-zero. The $DEADZONE = 6$ count threshold prevents limit cycling near the setpoint. Together, these two mechanisms form a practical solution to the common problem of motor control near the setpoint in low-budget hardware with significant Coulomb friction.

6.3 Iterative Decision-Making Process

Table 6.1: Iterative decision-making process.

Objectives	Problems	Iterative Process	Decision Making	Achieved/Overcome	Improvements Needed
Build hardware prototype	Motor direction inconsistent	Approach 1: Swap motor wires. Approach 2: Negate encoder counter in firmware.	Approach 2 selected: more robust and software-controllable without re-wiring.	Consistent direction mapping confirmed.	PCB layout for noise reduction.
Implement digital PID	Large K_d discrete (91.17) caused oscillation	Approach 1: Apply $K_d = 91.17$ directly. Approach 2: Reduce to $K_d = 3.0$ empirically.	Approach 2 selected: MATLAB warning confirmed noise amplification risk.	Stable operation achieved with $K_d = 3.0$.	Implement derivative filter for better K_d utilization.
Achieve 90-degree positioning	Motor stalls at small errors (stiction)	Approach 1: Increase K_p . Approach 2: Add PWM offset of 40.	Approach 2 selected: pure gain increase would reduce	Motor reliably reaches target within deadzone.	Adaptive offset based on load estimation.

			stability margin.		
--	--	--	----------------------	--	--

CHAPTER VII - CONCLUSION AND FUTURE WORK

7.1. Conclusions

This project successfully designed and implemented a complete closed-loop digital motion control system for precise angular positioning of a DC motor at 90 degrees using an STM32F407 microcontroller. The following conclusions are drawn:

- The hardware prototype, consisting of an STM32F4 Discovery board, DC motor with quadrature encoder, and L298N H-bridge driver, was successfully assembled and operated reliably.
- Open-loop system identification using MATLAB's System Identification Toolbox yielded an accurate second-order transfer function $G(s) = 8.9958 / (s^2 + 3.9716s)$ with a validation fit of 99.51%.
- Automated PID tuning via MATLAB's `pidentune()` identified optimal continuous parameters ($K_p = 4.1671$, $K_i = 4.2523$, $K_d = 0.9117$) that, after discretization, were successfully implemented in STM32 firmware.
- The optimized controller achieved a simulation settling time of 1.90 seconds (vs. 3.73 s for manual tuning) with 9.3% overshoot, demonstrating a 49% improvement in transient response speed.
- Real-time hardware validation using Teleplot confirmed stable bidirectional position control with steady-state error within ± 6 encoder counts across multiple cycles.

7.2. Future Work

To further advance this robotic platform, several improvements are proposed:

- **Custom PCB Design:** Transitioning from testboards to a custom-designed Printed Circuit

Board (PCB) using Altium Designer to further eliminate EMI noise and improve current delivery.

- **Trajectory Planning:** Implementing trapezoidal or S-curve motion profiling to generate smooth, continuous target paths rather than abrupt step-inputs, reducing mechanical stress on the gearbox.
- **Advanced Control Algorithms:** Exploring State-Space control (LQR) or Fuzzy Logic Control to handle varying payload masses dynamically.

REFERENCES

- [1] K. J. Astrom and T. Hagglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.
- [2] L. Ljung, System Identification: Theory for the User, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
- [3] T. Soderstrom and P. Stoica, System Identification. London, UK: Prentice Hall International, 1989.
- [4] STMicroelectronics, "UM1472 User Manual: Discovery kit with STM32F407VG MCU," STMicroelectronics, Geneva, Switzerland, Rev. 7, 2023.
- [5] STMicroelectronics, "RM0090 Reference Manual: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs," STMicroelectronics, Geneva, Switzerland, Rev. 19, 2023.
- [6] MathWorks, "System Identification Toolbox User's Guide," The MathWorks, Inc., Natick, MA, 2024.
- [7] MathWorks, "Control System Toolbox User's Guide: pidtune function," The MathWorks, Inc., Natick, MA, 2024.
- [8] G. F. Franklin, J. D. Powell, and M. L. Workman, Digital Control of Dynamic Systems, 3rd ed. Menlo Park, CA: Addison-Wesley, 1997.
- [9] W. Levine, Ed., The Control Handbook: Control System Fundamentals, 2nd ed. Boca Raton, FL: CRC Press, 2010.
- [10] Teleplot, "Teleplot – Open-source remote telemetry and plotting tool," [Online]. Available: <https://teleplot.fr>. [Accessed: Jun. 2026].

APPENDICES

APPENDIX A: EMBEDDED FIRMWARE IMPLEMENTATION (STM32)

Appendix A.1: Peripheral Initialization and Configuration

The system initializes TIM1 in PWM mode for motor speed regulation, TIM2 in Encoder mode for position feedback, and USART2 for real-time telemetry. The system clock is configured for a 168MHz operation to ensure high-speed processing of the PID loop.

Appendix A.2: PID Control Loop and FSM Logic

The core logic resides within the while(1) main loop. It consists of a 10ms sampling interval, a four-state FSM for cycle automation, and a discrete PID algorithm with anti-windup and stiction compensation.

```
/* USER CODE END 0 */

/* Inside main while(1) loop */
while (1)
{
    // 1. Read quadrature encoder position
    encoder_value = -(int32_t) __HAL_TIM_GET_COUNTER(&htim2);

    // 2. Finite State Machine (FSM) for automated cycle control
    switch (currentState)
    {
        case MOVE_TO_90:
            target = 515;
```

```

    if (abs(target - encoder_value) <= DEADZONE) {

        control_motor(0);

        integral = 0; last_error = 0;

        state_timer = HAL_GetTick();

        currentState = WAIT_AT_90;

    }

    break;

case WAIT_AT_90:

    control_motor(0);

    if (HAL_GetTick() - state_timer > 3000) { // 3s dwell time

        integral = 0; last_error = 0;

        currentState = MOVE_TO_0;

    }

    break;

case MOVE_TO_0:

    target = 0;

    if (abs(target - encoder_value) <= DEADZONE) {

        control_motor(0);

        integral = 0; last_error = 0;

        state_timer = HAL_GetTick();

        currentState = WAIT_AT_0;

```

```

    }

    break;

case WAIT_AT_0:

    control_motor(0);

    if (HAL_GetTick() - state_timer > 3000) {

        integral = 0; last_error = 0;

        currentState = MOVE_TO_90;

    }

    break;
}

// 3. PID Control Algorithm (executed only in MOVE states)
if (currentState == MOVE_TO_90 || currentState == MOVE_TO_0)
{
    error = target - encoder_value;

    pPart = Kp * (float)error;

    // Anti-windup: accumulate integral only near setpoint
    if (abs(error) < 100) integral += (float)error;

    else integral = 0;

    // Integral clamping

```

```

    if (integral > 200) integral = 200;

    if (integral < -200) integral = -200;


    iPart = Ki * integral;

    dPart = Kd * (float)(error - last_error);

    last_error = error;


    out_pwm = (int16_t)(pPart + iPart + dPart);


    // Apply PWM offset to overcome static friction (stiction)
    if (abs(error) > DEADZONE) {
        if (out_pwm > 0) out_pwm += PWM_OFFSET;

        else if (out_pwm < 0) out_pwm -= PWM_OFFSET;
    }


    // Output clamping
    if (out_pwm > 500) out_pwm = 500;

    if (out_pwm < -500) out_pwm = -500;


    control_motor(out_pwm);
}


// 4. Real-time telemetry via UART for Teleplot

```

```

int len = sprintf(msg, ">Target:%ld\n>Encoder:%ld\n>State:%d\n>Error:%d\n>PWM:%d\n",
    target, encoder_value, (int)currentState, error, out_pwm);

HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 10);

HAL_Delay(10); // Fixed 10ms sampling period
}

```

APPENDIX B: MATLAB SYSTEM IDENTIFICATION AND OPTIMIZATION

Appendix B.1: Data Preprocessing and System Identification

The script below performs the cleaning of raw UART telemetry data, filters the open-loop transient response, and estimates the transfer function model using the tfest algorithm.

```

% 1. Load and Clean Raw Data

T_raw = readtable('sysid_data.csv');

% Forward-fill missing values caused by asynchronous UART transmission

pwm_col = T_raw.SysID_PWM;
enc_col = T_raw.SysID_Enc;

% ... (Insert loop logic here)


% 2. Identification of Open-Loop Transfer Function

% Segmenting the dataset for training and validation

n_tr = round(0.6 * n);

d_train = iddata(enc_filt(1:n_tr), pwm_filt(1:n_tr), Ts);

```

```
% Estimating 2nd order transfer function (2 poles, 0 zeros)

sys20 = tfest(d_train, 2, 0, opt);

% Displaying best model fit (typically >99% for this system)

disp(sys20);
```

Appendix B.2: Automated PID Tuning and Discretization

This script computes the optimal PID gains by performing a grid search over frequency-domain constraints (Crossover Frequency and Phase Margin) and discretizes the resulting parameters for the STM32 C-code.

```
% 1. Grid Search for Optimal PID parameters

for cf = [2, 4, 6, 8] % Crossover Frequency (rad/s)

    for pm = [50, 60, 70] % Phase Margin (degrees)

        opts = pidtuneOptions('CrossoverFrequency', cf, 'PhaseMargin', pm);

        C = pidtune(best, 'pid', opts);

        % Evaluate stability and performance (SettlingTime < 2s)

        % ... (Insert verification logic)

    end

end

% 2. Discrete Parameter Conversion for Embedded Deployment

% The STM32 PID loop runs at Ts = 10ms

Kp_code = C_best.Kp;

Ki_code = C_best.Ki * Ts;

Kd_code = C_best.Kd / Ts;
```

```
fprintf('float Kp = %.4ff; float Ki = %.4ff; float Kd = %.4ff;\n', ...  
       Kp_code, Ki_code, Kd_code);
```